

Übersicht

–

Level

1

1. Mikrocontroller verstehen – Begriffserklärung	3
2. Installation der Arduino IDE	3
3. Vorwort zu den Programmierübungen.....	3
4. Programmierübung Level: 1.1.....	4
5. Programmierübung Level: 1.2.....	5
6. Programmierübung Level: 1.3.....	6
7. Programmierübung Level: 1.4.....	8

1. Mikrocontroller verstehen – Begriffserklärung

Was ist ein Mikrocontroller, wofür werden sie eingesetzt und wo liegt der Unterschied zum Computer? Das sind die Ersten drei Fragen mit denen wir uns befassen bevor wir in diesem Abschnitt die Zentralen Begriffe rund um Mikrocontroller Klären.

2. Installation der Arduino IDE

Hier lernst du wie du die Arduino IDE installierst und einrichtest.

3. Vorwort zu den Programmierübungen

Die Programmierübungen sind so aufgebaut, dass sie Schritt für Schritt in die Arbeit mit Mikrocontrollern einführen.

Alle Übungen können mit dem Hardware-Einsteigerpaket Level 1 umgesetzt werden und bauen inhaltlich aufeinander auf.

Zu Beginn werden grundlegende Konzepte vermittelt und einfache Aufgaben umgesetzt. Mit jeder weiteren Übung werden neue Funktionen ergänzt und bestehende Inhalte erweitert.

Ziel ist es, ein solides Verständnis für den Aufbau von Programmen, den Umgang mit der Arduino IDE und das Zusammenspiel von Hard- und Software zu entwickeln.

Die Übungen sind bewusst übersichtlich gehalten und sollen dazu ermutigen, eigene Anpassungen vorzunehmen und mit dem Code zu experimentieren.

Es wird empfohlen, die Programmierübungen in der vorgesehenen Reihenfolge durchzuführen, da spätere Aufgaben auf den vorherigen aufbauen.

So entsteht nach und nach ein nachvollziehbarer Lernpfad, der als Grundlage für weiterführende Projekte dient.

4. Programmierübung Level: 1.1

In dieser Programmierübung setzt du die zuvor vermittelten Grundlagen erstmals praktisch um.

Anhand eines einfachen Beispiels lernst du, wie ein Arduino-Programm aufgebaut ist und wie ein Mikrocontroller mit der Arduino IDE programmiert wird.

Du lernst:

- wie ein Arduino-Sketch aufgebaut ist (setup() und loop())
- wie ein digitaler Ausgang konfiguriert wird
- wie du eine LED gezielt ein- und ausschaltest
- wie Zeitverzögerungen im Programm verwendet werden
- wie du ein Programm auf einen Arduino Nano überträgst

Am Ende der Übung läuft das Programm selbstständig auf dem Mikrocontroller und die eingebaute LED blinkt regelmäßig.

Diese Übung bildet die Grundlage für alle weiteren Programmieraufgaben in Level 1.

5. Programmierübung Level: 1.2

In dieser Programmierübung wird die bestehende Schaltung um eine zweite externe LED erweitert.

Aufbauend auf den Grundlagen aus Level 1.1 lernst du, wie mehrere digitale Ausgänge gleichzeitig angesteuert und logisch miteinander verknüpft werden.

Der Fokus liegt darauf, bestehende Programme sinnvoll zu erweitern und unterschiedliche Blinkmuster umzusetzen, ohne die Grundstruktur des Programms zu verändern.

Du lernst:

- wie mehrere LEDs an unterschiedlichen Pins angesteuert werden
- wie mehrere Ausgänge gleichzeitig geschaltet werden können
- wie zeitgesteuerte Abläufe für mehrere Komponenten umgesetzt werden
- wie Programme Schritt für Schritt erweitert werden

Umgesetzt werden folgende Aufgaben:

- zwei LEDs blinken abwechselnd
- zwei LEDs blinken gleichzeitig

Bonusaufgabe:

- zwei LEDs blinken mit unterschiedlichen Blinkgeschwindigkeiten

Anhand der Bonusaufgabe wird gezeigt, dass zeitgesteuerte Abläufe mit einfachen Verzögerungen (`delay()`) zwar möglich sind, bei mehreren parallelen Abläufen jedoch schnell unübersichtlich werden.

Diese Erkenntnis dient als Vorbereitung für spätere Übungen, in denen zeitgesteuerte Abläufe strukturiert und nicht blockierend umgesetzt werden.

Für diese Übung werden folgende Pins verwendet:

- LED 1 → D5
- LED 2 → D6

Am Ende der Übung laufen beide LEDs selbstständig auf dem Arduino Nano und reagieren entsprechend dem programmierten Blinkmuster.

Diese Programmierübung bildet die Grundlage für alle weiteren Aufgaben, in denen mehrere Ausgänge gleichzeitig gesteuert und später durch Eingaben beeinflusst werden.

6. Programmierübung Level: 1.3

In dieser Programmierübung wird die Schaltung um zwei Taster als digitale Eingänge erweitert.

Die LEDs werden nun nicht mehr ausschließlich zeitgesteuert, sondern reagieren auf Benutzereingaben.

Ziel dieser Übung ist es, das Zusammenspiel von Eingängen (Taster), Programmlogik und Ausgängen (LEDs) zu verstehen und erste zustandsabhängige Abläufe umzusetzen.

Du lernst:

- wie digitale Eingänge gelesen werden
- wie Taster zur direkten Steuerung von Ausgängen verwendet werden
- wie einfache Schaltlogiken umgesetzt werden
- wie Programme auf Benutzereingaben reagieren, ohne neu gestartet zu werden

Standardaufgaben

- Taster 1 schaltet LED 1 ein, solange er gedrückt wird
- Taster 2 schaltet LED 2 ein, solange er gedrückt wird
- Taster 1 schaltet beide LEDs ein
- Taster 2 schaltet beide LEDs aus
- Taster 1 schaltet LED 1 ein und LED 2 aus
- Taster 2 schaltet LED 2 ein und LED 1 aus

Diese Aufgaben vermitteln ein sicheres Verständnis für:

- digitale Eingaben
- logische Verknüpfungen
- eindeutige Zustände von Ausgängen

Bonusaufgabe

- Taster 1 startet ein abwechselndes Blinkmuster der LEDs
- Taster 2 beendet das Blinkmuster

Anhand dieser Aufgabe wird deutlich, dass zeitgesteuerte Abläufe und gleichzeitige Eingaben mit einfachen Verzögerungen (`delay()`) nur eingeschränkt sinnvoll umgesetzt werden können.

Zur Lösung dieser Aufgabe wird erstmals eine nicht blockierende Zeitsteuerung eingeführt, die es erlaubt, Blinkmuster und Tasterabfragen gleichzeitig auszuführen.

Challenge (optional)

- Auswahl verschiedener Blinkrhythmen über die Taster
- Anpassung der Blinkgeschwindigkeit durch wiederholtes Betätigen der Taster
- gleichzeitiges Drücken beider Taster schaltet alle LEDs aus

Diese Challenge erfordert:

- strukturierte Programmlogik
- sauberes Zustandsdenken
- kontrollierte zeitgesteuerte Abläufe

Für diese Übung werden folgende Pins verwendet:

- LED 1 → D5
- LED 2 → D6
- Taster 1 → D7
- Taster 2 → D8

Die Taster werden mit aktivierter interner Pull-Up-Widerstandsschaltung betrieben (INPUT_PULLUP).

Am Ende der Übung reagiert das Programm zuverlässig auf Benutzereingaben, steuert die LEDs abhängig vom aktuellen Zustand und bleibt jederzeit bedienbar.

Diese Programmierübung bildet den Übergang von einfachen zeitgesteuerten Abläufen zu strukturierter Programmlogik und bereitet gezielt auf die Inhalte von Level 2 vor.

7. Programmierübung Level: 1.4

In dieser Programmierübung wird die Schaltung um ein Potentiometer als analogen Eingang erweitert.

Erstmals werden kontinuierliche Messwerte verarbeitet und zur dynamischen Steuerung des Programmablaufs genutzt.

Ziel dieser Übung ist es, analoge Eingänge zu verstehen und deren Messwerte sinnvoll in sichtbare Veränderungen umzusetzen.

Du lernst:

- wie analoge Eingänge ausgelesen werden
- wie analoge Messwerte interpretiert und weiterverarbeitet werden
- wie Programmabläufe dynamisch beeinflusst werden können
- wie zeitgesteuerte Abläufe mit variablen Parametern umgesetzt werden

Standardaufgabe

- das Potentiometer steuert die Helligkeit der LEDs

Hierbei wird die Helligkeit über Pulsweitenmodulation (PWM) geregelt.

Es wird gezeigt, dass keine echte Analogspannung ausgegeben wird, sondern ein schnelles Ein- und Ausschalten des Ausgangs genutzt wird, um unterschiedliche Helligkeiten zu erzeugen.

Für diese Übung werden folgende Pins verwendet:

- LED 1 (PWM) → D5
- LED 2 (PWM) → D6
- Potentiometer → A0

Nicht alle Pins des Arduino Nano unterstützen PWM.

Aus diesem Grund werden für diese Übung bewusst die Pins D5 und D6 verwendet.

Am Ende der Übung kann der Programmablauf während der Laufzeit flexibel beeinflusst werden.

Die LEDs reagieren dynamisch auf Änderungen am Potentiometer, ohne dass das Programm neu gestartet werden muss.

Diese Programmierübung schließt Level 1 ab und bildet die Grundlage für die Verarbeitung von Sensorwerten, die Anzeige von Informationen auf Displays und komplexere Steuerungen in Level 2.